dabeer.ai

5 skills that keep engineers hired

Layoffs are everywhere, but AI hiring in India is growing. None of these skills is "just prompting": companies want engineers who can **ship and run AI**. This guide is a 10-week plan, with projects, interview questions and free resources for each skill.

+25%
AI & ML hiring on Naukri, year on year

−3%
IT services hiring, same period

10
weeks, 2 per skill, 1 portfolio project

THE 10-WEEK PATH

- 🔎 **Querying data** — WEEKS 1–2
- 🧠 **Building on LLMs** — WEEKS 3–4
- 📈 **LLMOps** — WEEKS 5–6
- ⚙️ **Workflow automation** — WEEKS 7–8
- 🔐 **Cloud + security for AI** — WEEKS 9–10

dabeer.ai

How to use this roadmap

Who it's for

Software, cloud, DevOps and data engineers with 0–8 years of experience who want to move into AI work, or stay safe as companies shift budgets to AI. You need basic programming (Python or JavaScript). No machine-learning maths.

Time

About 1 hour on weekdays and 3–4 hours at weekends: roughly 8–10 hours a week. Ten weeks if you go one skill every two weeks; take longer if you need to, but don't skip the projects.

Setup (day 0)

- Python 3 or Node.js, Git and a GitHub account
- PostgreSQL (Docker is easiest)
- An LLM API key with a spending limit set
- A free-tier cloud account

The 4 rules

- **Build, don't just watch.** Every skill ends with a project; tutorials alone won't get you hired.
- **One GitHub repo for the whole journey.** Each skill adds to the same app, so week 10 ends with one strong portfolio project.
- **Write down numbers.** Eval scores, cost per answer, hours saved. Numbers go on your CV.
- **Share progress publicly** on LinkedIn every two weeks. Recruiters search for exactly these skills.

How the skills connect

You query data (1) to build a docs assistant (2), run it in production (3), connect it to real work (4) and make it safe (5). By week 10 it's one app, not five toy demos.

dabeer.ai

1 🔎 Querying data

Every AI feature starts with data. Engineers who can answer questions with SQL get pulled into AI work first.

What it is

Pulling exactly the data you need out of a database, joining it, summarising it, and turning it into an answer someone can act on. In AI work, this is how you build datasets for RAG, check what your model actually did from logs, and measure whether a feature helped.

Why companies pay for it

- LinkedIn's 2026 list ranks querying as the fastest-growing engineering skill in India.
- Every RAG, analytics and evals pipeline sits on top of a database.
- It's the quickest skill to show proof of: a notebook of real answers.

Your 2-week plan (~1 hour a day, weekends for the project)

DAY 1-2	SELECT, WHERE, ORDER BY, LIMIT on one table. Do every SQLBolt lesson.
DAY 3-4	GROUP BY, HAVING, COUNT/SUM/AVG. Answer: "top 10 products by revenue per month".
DAY 5-6	JOINs (inner, left), and why a wrong join doubles your numbers.
DAY 8-9	CTEs (WITH ...) to break a big query into readable steps.
DAY 10-11	Window functions: ROW_NUMBER, RANK, running totals, LAG for "change vs last month".
DAY 12-14	Indexes and EXPLAIN: why a query is slow, and how to fix it. Build the mini-project.

dabeer.ai

Mini-project: Business questions notebook

Load a public dataset (e.g. an e-commerce or ride dataset from Kaggle) into PostgreSQL and answer 10 real business questions.

DONE WHEN

- ☐ 10 questions, each with its query and a one-line answer
- ☐ At least 3 use window functions, 2 use CTEs
- ☐ One slow query fixed with an index, with EXPLAIN before and after
- ☐ README on GitHub with the questions and findings

Practice exercises

- Find each customer's first and latest order in one query.
- Month-over-month growth % with LAG.
- Deduplicate rows keeping the latest record per ID.

Common mistakes

- Joining before filtering, then wondering why numbers are inflated.
- SELECT * in production code.
- Not checking NULLs: COUNT(col) skips them, COUNT(*) does not.

Interview questions you'll get

Q. WHERE vs HAVING?
WHERE filters rows before grouping; HAVING filters groups after aggregation.

Q. How would you find duplicate emails?
GROUP BY email HAVING COUNT(*) > 1, or ROW_NUMBER() OVER (PARTITION BY email) to keep one.

Q. A query got slow overnight. What do you check?
EXPLAIN plan, missing or unused indexes, table growth, stale statistics, and a changed filter.

Learn free

SQLBolt	sqlbolt.com	Interactive basics, 1–2 hours
Mode SQL Tutorial	mode.com/sql-tutorial	Basic to advanced, with real datasets
PostgreSQL Exercises	pgexercises.com	Practice joins, aggregates, windows
Use The Index, Luke	use-the-index-luke.com	Indexes and performance, free

2 🧠 Building on LLMs

Calling a model takes 5 lines. Making it answer correctly from your company's data, every time, is the job.

What it is

Building applications on top of large language models: calling the API, giving the model the right context (RAG), letting it use tools (agents), and testing that answers are right (evals).

Why companies pay for it

- On Naukri, AI and ML hiring is up 25% in a year while IT services hiring fell 3%.
- Most company AI projects are RAG and agents, not training new models.
- Evals are what separate a demo from something a company will ship.

Your 2-week plan (~1 hour a day, weekends for the project)

DAY 1-2	Call an LLM API (Claude, GPT or Gemini). Stream the answer. Handle errors and timeouts.
DAY 3	System prompts and structured output (JSON). Make the model reply in a fixed format.
DAY 4-6	RAG: split documents into chunks, create embeddings, store them in a vector database (pgvector or Chroma), retrieve the top matches and answer with sources.
DAY 8-10	Tools / agents: give the model 2–3 functions (search orders, create ticket). Learn MCP, the standard way to connect tools.
DAY 11-12	Evals: 20 real questions with expected answers; score your app automatically.
DAY 13-14	Build the mini-project and improve it until the eval score goes up.

Mini-project: "Chat with your docs" with sources

A web app where you upload PDFs (e.g. your company's public policies or a product manual) and ask questions; every answer cites the page it came from.

DONE WHEN

- ☐ Answers cite the source document and page
- ☐ Says "I don't know" when the answer isn't in the documents
- ☐ An eval set of 20 questions with a score you can show
- ☐ Deployed somewhere with a link in your README

Practice exercises

- Make the model return valid JSON 50 times in a row.
- Compare chunk sizes (200 vs 800 tokens) on the same eval set.
- Add a tool and make the model refuse when it lacks permission.

Common mistakes

- Skipping evals: you can't tell if a change made things better or worse.
- Stuffing whole documents into the prompt instead of retrieving.
- Trusting the model's JSON without validating it.

Interview questions you'll get

Q. RAG or fine-tuning for company knowledge?
RAG for facts that change and need sources; fine-tuning for style or format. Most teams start with RAG.

Q. How do you reduce hallucinations?
Ground answers in retrieved documents, ask for citations, allow "I don't know", and measure with evals.

Q. What is an agent?
A model that decides which tools to call, in a loop, until the task is done, with limits on steps and permissions.

Learn free

Anthropic docs & cookbook	`docs.anthropic.com` · `github.com/anthropics`	Prompting, tools, RAG examples
OpenAI docs & cookbook	`platform.openai.com/docs`	API, structured outputs, embeddings
Model Context Protocol	`modelcontextprotocol.io`	The standard for connecting tools
pgvector	`github.com/pgvector/pgvector`	Vector search inside PostgreSQL

dabeer.ai

3 📈 LLMOps

A prompt that worked on Monday can quietly break on Friday. LLMOps is how you find out before your users do.

What it is

Running AI applications in production: deploying them, tracing every request, tracking cost and latency, versioning prompts, and running evals automatically on every change, the way DevOps does for normal code.

Why companies pay for it

- AI bills grow fast; companies need people who can see and control cost per feature.
- Model upgrades change behaviour; versioned prompts and evals catch regressions.
- It's the natural next step for DevOps and cloud engineers.

Your 2-week plan (~1 hour a day, weekends for the project)

DAY 1-2	Containerise your week 3–4 app and deploy it (any cloud or a free tier).
DAY 3-4	Add tracing (Langfuse or LangSmith): every prompt, retrieval, tool call and answer.
DAY 5	Track tokens, cost and latency per request; find your most expensive question.
DAY 8-9	Prompt versioning: store prompts in Git, tag versions, log which version answered.
DAY 10-11	Run your eval set in CI (GitHub Actions) on every pull request; fail the build if the score drops.
DAY 12-14	Caching and routing: cache repeated questions, send easy ones to a cheaper model. Build the dashboard.

dabeer.ai

Mini-project: AI ops dashboard

Production setup for your docs app: traces, cost and latency per request, prompt versions, and evals that run on every change.

DONE WHEN

- ☐ Every request traced end to end
- ☐ Dashboard: cost/day, p95 latency, eval score over time
- ☐ CI blocks a merge when the eval score drops
- ☐ A cache or model router with measured savings

Practice exercises

- Cut cost per answer by 30% without the eval score dropping.
- Make CI fail on a deliberately bad prompt change.
- Alert when p95 latency goes above 5 seconds.

Common mistakes

- Logging full prompts with customers' personal data in them.
- Upgrading the model version without re-running evals.
- Measuring average latency instead of p95.

Interview questions you'll get

Q. How do you know a prompt change didn't break anything?
An eval set of real questions, run automatically in CI, compared with the previous score.

Q. How would you reduce LLM cost?
Cache repeated answers, route simple questions to a smaller model, cap tokens, trim retrieved context.

Q. What do you log for an AI request?
Prompt version, retrieved chunks, tool calls, tokens, cost, latency and the answer, with personal data masked.

Learn free

Langfuse	langfuse.com/docs	Open-source tracing, evals, prompt versions
LangSmith	docs.smith.langchain.com	Tracing and evals
GitHub Actions	docs.github.com/actions	Run evals on every pull request
OpenTelemetry	opentelemetry.io	The standard for traces and metrics

4 ⚙️ Workflow automation

The engineer who saves their team 10 hours a week gets remembered at appraisal time.

What it is

Connecting AI to the tools a team already uses (email, Slack, Jira, Sheets, GitHub) so that real work happens without someone doing it by hand: triaging tickets, writing first drafts, summarising, routing.

Why companies pay for it

- It turns AI from a demo into hours saved, a number managers understand.
- Low-code tools (n8n) plus a little code cover most company workflows.
- Every team has a boring weekly task waiting to be automated.

Your 2-week plan (~1 hour a day, weekends for the project)

DAY 1-2	List 5 repetitive tasks in your team; estimate hours per week for each. Pick the biggest.
DAY 3-4	Learn n8n (self-hosted or cloud): triggers, nodes, credentials, error handling.
DAY 5-6	Add an AI step: classify, extract fields, or draft a reply, with a JSON output you validate.
DAY 8-9	Add a human-in-the-loop step: approve in Slack or email before anything is sent.
DAY 10-11	Handle failures: retries, alerts, a dead-letter list for items it couldn't process.
DAY 12-14	Roll it out to your team; measure time saved for one week.

dabeer.ai

Mini-project: One automation your team actually uses

A workflow that takes a real recurring task off someone's plate, with an AI step, approval before anything goes out, and a measured result.

DONE WHEN

- ☐ Runs on a real trigger (email, form, ticket, schedule)
- ☐ AI output is validated; bad output goes to a review queue
- ☐ A human approves before anything external is sent
- ☐ A before/after number: hours saved per week

Practice exercises

- Auto-label incoming support emails into 5 categories.
- Summarise a long Slack thread into decisions and owners.
- Turn a bug report into a structured Jira ticket.

Common mistakes

- Automating a process nobody actually does every week.
- Letting the AI send emails to customers without review.
- Storing API keys inside the workflow in plain text.

Interview questions you'll get

Q. How do you make an AI automation safe?
Validate outputs, keep a human approval step for external actions, log everything, and limit permissions.

Q. What happens when the AI step fails?
Retry with backoff, then send the item to a review queue and alert someone; never drop it silently.

Q. How do you prove the value?
Measure time per task before and after, and the error rate, over at least a week.

Learn free

n8n docs	`docs.n8n.io`	Workflows, AI nodes, self-hosting
Zapier / Make	`zapier.com · make.com`	If your company already uses them
Slack API	`api.slack.com`	Approval buttons and notifications
Your cloud's functions	`AWS Lambda / Azure Functions / Cloud Run`	For steps that need code

5 🔐 Cloud + security for AI

Every AI agent is a new identity with access to your systems. Most teams forget to treat it like one.

What it is

Giving AI systems their own identities, the least access they need, secrets that never sit in code or prompts, and network rules that stop data from leaking out. This is where cloud and security engineers have a big advantage.

Why companies pay for it

- Leaked API keys and over-permissioned agents are among the most common AI incidents.
- Banks and large companies won't ship AI without access control and audit logs.
- Few engineers know both AI and cloud security, so the skill is scarce.

Your 2-week plan (~1 hour a day, weekends for the project)

DAY 1-2	IAM on one cloud (AWS, Azure or GCP): users, roles, policies, least privilege.
DAY 3-4	Give your app its own role; remove every permission it doesn't use.
DAY 5-6	Secrets: move every key into a secrets manager (Vault, AWS Secrets Manager, Azure Key Vault).
DAY 8-9	Short-lived credentials: tokens that expire in minutes, issued per task.
DAY 10-11	Pass the user's identity through RAG and tools so the AI only sees what the user may see.
DAY 12-14	Network: private endpoints, restrict outbound traffic. Read the OWASP Top 10 for LLM apps.

Mini-project: Lock down your AI app

Take your docs app and make it production-safe: its own identity, least privilege, no secrets in code, per-user access in retrieval, and an audit log.

DONE WHEN

- ☐ Zero secrets in code, config or prompts
- ☐ App role has only the permissions it uses
- ☐ User A cannot retrieve User B's documents (tested)
- ☐ Audit log of every tool call with who asked

Practice exercises

- Search your repos for leaked keys (git history too) and rotate them.
- Write a policy that lets the app read one bucket and nothing else.
- Try a prompt injection on your own app and block it.

Common mistakes

- One admin API key shared by every service.
- Enforcing access in the prompt ("don't show salaries") instead of the backend.
- Keys that never expire.

Interview questions you'll get

Q. Where should an AI agent's API keys live?
In a secrets manager, issued short-lived and per task; never in code, prompts or logs.

Q. How do you stop an AI assistant leaking data across users?
Pass the user's identity into retrieval and tools and enforce access in the backend; also filter outputs.

Q. What is prompt injection?
Untrusted text that tries to override the AI's instructions; defend with input checks, least privilege and output filters.

Learn free

OWASP Top 10 for LLM apps	genai.owasp.org	The main AI security risks
HashiCorp Vault tutorials	developer.hashicorp.com/vault/tutorials	Secrets and dynamic credentials
AWS IAM user guide	docs.aws.amazon.com/IAM	Roles, policies, least privilege
Azure Key Vault / GCP Secret Manager	learn.microsoft.com · cloud.google.com	If you're on Azure or GCP

dabeer.ai

Your capstone project

One app that uses all five skills. This is what you show in interviews.

User signs in (SSO)	**Your app** API + chat UI	**Identity check** role, team, permissions
RAG pgvector over company docs	**LLM API** Claude / GPT / Gemini	**Tools (MCP)** tickets, orders, SQL
Secrets vault short-lived keys	**Tracing + evals** Langfuse, CI	**Automation** n8n workflow + approval

Done when

- ☐ Answers questions from your documents with citations, and says "I don't know" otherwise
- ☐ Can run at least one real action through a tool, with approval for anything external
- ☐ User A can never see User B's documents (and you have a test proving it)
- ☐ No secrets in code; every key comes from a vault and expires
- ☐ Dashboard of cost, latency and eval score; CI blocks a merge if the score drops
- ☐ A README with an architecture diagram, numbers and a 2-minute demo video

Résumé bullets you can write after this

- Built a RAG assistant over 200+ documents with cited answers; raised eval accuracy from 62% to 88%.
- Cut LLM cost per answer 35% with caching and model routing, tracked in a Langfuse dashboard.
- Secured an AI agent with least-privilege IAM, short-lived credentials and per-user access in retrieval.
- Automated ticket triage with an approval step, saving the team ~6 hours a week.

Use your real numbers. These are examples of the format.

dabeer.ai

Are you job-ready?

- ☐ I can answer business questions with SQL, including window functions
- ☐ I've built RAG with citations and measured it with an eval set
- ☐ I've built an agent with tools and limits on what it can do
- ☐ My app is deployed, traced, and I know its cost per answer
- ☐ My evals run automatically on every change
- ☐ I've automated one real task and measured the time saved
- ☐ My app has its own identity, least privilege and no secrets in code
- ☐ I can explain prompt injection and how I defend against it
- ☐ My GitHub README tells the story with numbers and a diagram
- ☐ I've posted my progress on LinkedIn at least 5 times

Interview day tips

- Lead with the capstone: problem, architecture, numbers, what broke and how you fixed it.
- When asked about AI risks, talk about access control, secrets and evals; few candidates do.
- Have your eval results and cost dashboard open, ready to share your screen.

Keep going

Follow **@dabeer.ai** for AI system design, cloud and security, explained by an engineer. Comment **DESIGN** on any AI System Design reel for the free cheat sheet.